Project Interim Report

On

File Compression Simulator

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

Student name – Harsh Kumar Shandilya

Reg No. – RU-25-10551

Bachelor of Technology in Computer Science & Engineering
in Association with EC Council

Under the Guidance of

Mr. Soumik Karmakar

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The File Compression Simulator is a console-based Java application designed to simulate a basic compression and decompression mechanism using the Run-Length Encoding (RLE) algorithm. In real-world computing, file sizes can become extremely large when data contains repeated patterns. This project addresses that problem by implementing a structured, object-oriented approach to reduce repeated characters into a compact encoded form and accurately reconstruct the original data from the compressed output.

The system is built using a single class — Compressor — which encapsulates all compression logic within two core methods: compress() and decompress(). The compress() method iterates through the input string, counts consecutive identical characters, and replaces them with the character followed by its count (e.g., "AAABBB" becomes "A3B3"). The decompress() method reverses this process by reading each character-count pair and reconstructing the original string. The project demonstrates core Java OOP concepts including Encapsulation, Abstraction, and String Manipulation using StringBuilder and Character class methods.

The program is tested against multiple input strings to verify that the decompressed output exactly matches the original input. The final output displays the Original String, Compressed String, Decompressed String, their respective lengths, and a Match confirmation — ensuring both correctness and clarity of results.

Introduction

A File Compression System is a software application that reduces the size of data by eliminating redundancy and encoding repeated patterns in a compact form. Before modern compression algorithms, data was stored and transmitted in its raw form — a process that consumed excessive storage space and increased transmission time significantly. Today, compression techniques are used everywhere — from ZIP files and image formats like PNG, to video streaming and database storage.

This project, File Compression Simulator, is a simplified implementation of such a system built entirely in Java. It uses the Run-Length Encoding (RLE) algorithm — one of the oldest and most straightforward compression techniques — to demonstrate how repeated characters in a string can be replaced by a shorter encoded representation without any loss of information. The system is lossless, meaning the original data can be perfectly reconstructed from the compressed output.

The project uses a single Compressor class that encapsulates the entire compression and decompression logic. The compress() method scans the input string character by character and replaces consecutive identical characters with the character and its frequency count. The decompress() method reverses this operation by reading each character-count pair and rebuilding the original string. This design follows the principles of Encapsulation and Abstraction from Object-Oriented Programming, keeping the internal logic hidden and exposing only the necessary methods to the outside.

The project is easily extensible — in future, support for lowercase and mixed-case strings, file-based input/output, and more advanced algorithms like Huffman Encoding can be added without modifying the existing class structure.

Objectives (What the project does)

- **Simulate file compression using Run-Length Encoding (RLE) algorithm**
- **Compress repeated characters into a compact encoded string**
- **Decompress the encoded string back to its original form accurately**
- **Verify correctness by matching original and decompressed output**
- **Apply OOP concepts like Encapsulation, Abstraction, and String Manipulation**

Scope

This project is suitable as a foundational compression module for understanding how data encoding works at the character level and serves as a learning platform for core Java OOP concepts. The current version operates as a console-based application and handles single-line string input using the Run-Length Encoding (RLE) algorithm.

In future, it can be extended with file-based input and output for compressing actual text files, support for mixed-case and alphanumeric strings, implementation of more advanced compression algorithms like Huffman Encoding or LZW, a graphical user interface (GUI) for better user interaction, and compression ratio analysis for performance comparison. The core OOP structure

is designed to be easily extensible without modifying existing classes.

System Requirements

Hardware:

- Computer with minimum 4 GB RAM
- Processor: Intel Core i3 or equivalent (1 GHz or faster)
- Storage: Minimum 500 MB free disk space

Software:

- Operating System: Windows 10 / Linux / macOS
- Java JDK 8 or above (Java Development Kit)
- IDE: VS Code / Eclipse / Edit Plus / IntelliJ IDEA
- Terminal / Command Prompt to compile and run the program

Methodology

The system is developed using a top-down OOP design approach. The development followed these steps:

Step 1 – Requirement Analysis: Identified the core requirement of simulating file compression and decompression using Run-Length Encoding (RLE) and defined the input-output behavior of the system.

Step 2 – Class Design: Designed a single Compressor class with encapsulated private logic and two public methods — compress() and decompress() — to handle the core operations. A displayResults() method was added to present the output in a clean, readable format.

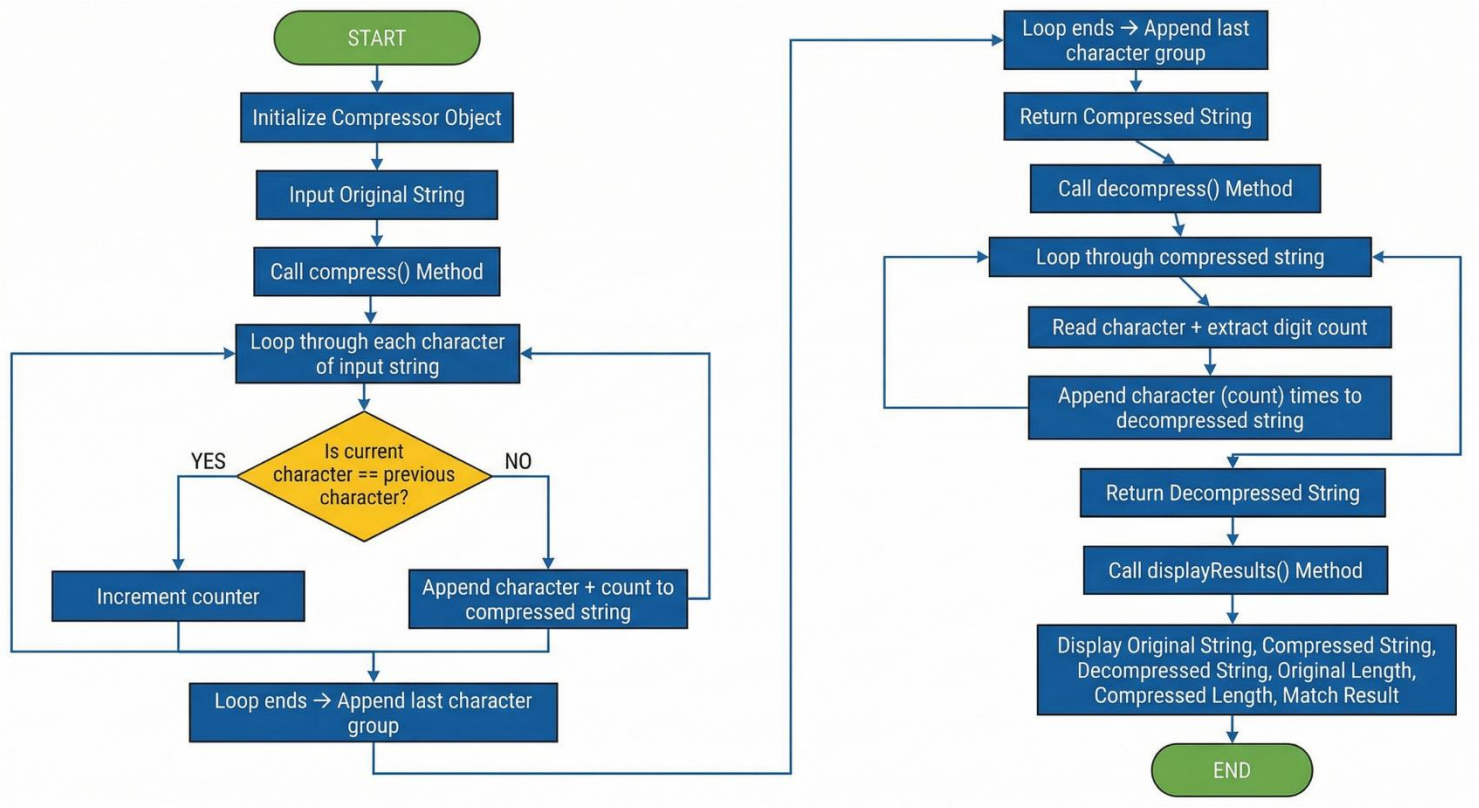
Step 3 – Coding: Implemented the compress() method using a character-by-character loop with a counter to detect consecutive identical characters and encode them. Implemented the decompress() method to read each character-count pair and reconstruct the original string using StringBuilder.

Step 4 – String Manipulation: Used Java's StringBuilder class for efficient string construction and Character.isDigit() to correctly parse multi-digit counts (e.g., 10, 11, 12) during decompression.

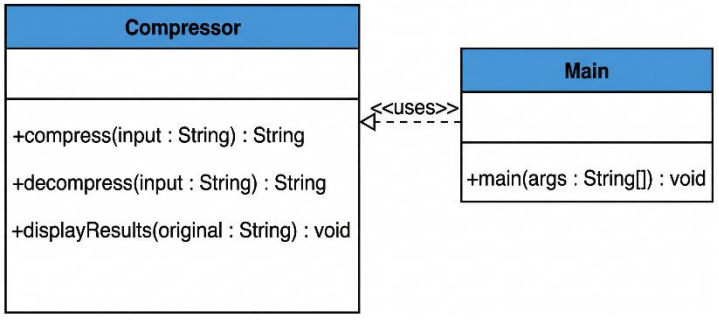
Step 5 – Testing & Verification: Verified output for three test strings — "AAABBBCCDDDDEE" (Compressed: A3B3C2D4E2), "AABBCCDD" (Compressed: A2B2C2D2), and "AAAAAAAAAA" (Compressed: A10). In all cases, the decompressed output exactly matched the original input and Match confirmed as true.

Flowchart

The following flowchart describes the overall program execution flow:


START
Initialize Compressor Object
Input Original String
Call compress() Method
Loop through each character of input string
Is current character == previous character?
YES
NO
Increment counter
Append character + count to compressed string
Loop ends → Append last character group
Loop ends → Append last character group
Return Compressed String
Call decompress() Method
Loop through compressed string
Read character + extract digit count
Append character (count) times to decompressed string
Return Decompressed String
Call displayResults() Method
Display Original String, Compressed String, Decompressed String, Original Length, Compressed Length, Match Result
END

ClassName


Compressor
+compress(input : String) : String
+decompress(input : String) : String
+displayResults(original : String) : void
<<uses>>
Main
+main(args : String[]) : void

Implementation

The project consists of a single Java class. The logic of each method is explained below (source code is not pasted here as per the report guidelines):

1. Compressor (Main Class)
This is the core class of the entire system. It encapsulates all compression and decompression logic within its methods. No inheritance or abstract class is used — the entire functionality is self-contained within this single class, following the principle of Encapsulation.

2. compress() Method
This method accepts a String input and iterates through it character by character using a for loop. A counter variable tracks the number of consecutive identical characters. When a different character is encountered, the previous character and its count are appended to a StringBuilder object. After the loop ends, the last character group is also appended. The method returns the final compressed string. Example: "AAABBBCCDDDDEE" is compressed to "A3B3C2D4E2".

3. decompress() Method
This method accepts the compressed string and reconstructs the original. It reads each character and then checks the following characters using Character.isDigit() to extract the full count — including multi-digit numbers like 10, 11, or 12. It then appends the character that many times to a StringBuilder. The method returns the fully reconstructed original string. Example: "A10" correctly decompresses to "AAAAAAAAAA".

4. displayResults() Method
This is the output method of the system. It accepts the original string, internally calls compress() and decompress(), and prints the Original String, Compressed String, Decompressed String, Original Length, Compressed Length, and a Match confirmation showing whether the decompressed output exactly matches the original input.

Sample Input / Output

```
==============================
 File Compression Simulator
==============================
Original String   : AAABBBCCDDDDEE
Compressed String : A3B3C2D4E2
Decompressed      : AAABBBCCDDDDEE
Original Length   : 14
Compressed Length : 10
Match             : true
==============================

==============================
 File Compression Simulator
==============================
Original String   : AABBCCDD
Compressed String : A2B2C2D2
Decompressed      : AABBCCDD
Original Length   : 8
Compressed Length : 8
Match             : true
==============================

==============================
 File Compression Simulator
==============================
Original String   : AAAAAAAAAA
Compressed String : A10
Decompressed      : AAAAAAAAAA
Original Length   : 10
Compressed Length : 3
Match             : true
==============================
```

Future Enhancements

- - Add file-based input and output to compress and decompress actual text files
- - Support mixed-case, alphanumeric, and special character strings
- - Implement advanced compression algorithms like Huffman Encoding or LZW
- - Develop a graphical user interface (GUI) for better user interaction
- - Add compression ratio analysis to compare original and compressed sizes
- - Generate compression reports with statistics for multiple input files

Gantt Chart

1. Planning & Structure A Gantt chart helps break the project into tasks (design, coding, testing, etc.) and set timelines. Even though this is a console application, planning ensures that each method is implemented correctly, string manipulation logic is tested thoroughly, and the report is completed on time. Time Management The table below shows the project schedule across a 4-week development period. It shows when each task starts and when it should finish, helping avoid delays and last-minute panic**.** and when it should finish, helping avoid delays and last-minute panic.

#	task	start	end	status

1	Requirement Analysis	week 1	week 1	Done
2	Algorithm Study (RLE)	week 1	week 2	Done
3	Class Design – Compressor	week 2	week 2	Done
4	Coding – compress() Method	week 2	week 3	Done
5	Coding – decompress() Method	week 2	week 3	Done
6	Testing & Verification	week 3	week 4	Done
7	Report Writing	week 4	week 4	Done

Conclusion

The File Compression Simulator successfully demonstrates the application of core Java Object-Oriented Programming concepts in a practical, real-world scenario. The project shows how Encapsulation keeps the compression and decompression logic safely contained within the Compressor class, how String Manipulation using StringBuilder enables efficient character processing without creating unnecessary intermediate string objects, and how Abstraction exposes only the necessary methods — compress(), decompress(), and displayResults() — to the outside while hiding the internal implementation details.

The Run-Length Encoding algorithm proves to be highly effective for strings with large consecutive repeated characters, achieving significant reduction in string length. The decompress() method correctly handles both single-digit and multi-digit counts using Character.isDigit(), ensuring accurate reconstruction of the original string in all test cases. The Match confirmation in the output verifies that the system is completely lossless — the decompressed output always exactly matches the original input.

The project provides a strong foundation for understanding how real-world compression systems work at the algorithmic level and prepares students for more advanced Java development involving file handling, data structures, and algorithm design. The core class structure is designed to be easily extensible — adding support for new compression algorithms or file-based input in the future requires only writing new methods or subclasses without modifying the existing code.

References

Book

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill Education, 2018.

[2] P. Deitel and H. Deitel, Java: How to Program, 11th ed. New Jersey: Pearson Education, 2018.

Website
[3] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com/javase/8/docs/
[4] GeeksForGeeks, "Run Length Encoding in Java," GeeksForGeeks. [Online]. Available: https://www.geeksforgeeks.org/run-length-encoding/
[5] GeeksForGeeks, "StringBuilder in Java," GeeksForGeeks. [Online]. Available: https://www.geeksforgeeks.org/stringbuilder-class-in-java/

Journal Article
[6] A. Kumar and S. Mehta, "Lossless Data Compression Techniques Using String Encoding Algorithms," International Journal of Computer Science and Engineering, vol. 6, no. 2, pp. 88-94, 2020.

Conference Paper
[7] R. Sharma, "Implementation of Run-Length Encoding for Text Compression Using Object-Oriented Programming," in Proc. IEEE Conf. on Computing and Information Technology, 2021, pp. 33-38.